

DEAD AIR

Quest design, dialogue tree and process notes

Stefan Petrovici | stefanpetrovici2006@gmail.com | firststef.github.io/narrative-portfolio

A quest from **Streamer UNTIL**, a first person Unity horror game by a four person team. The player is an urban explorer livestreaming a derelict theatre, and the antagonist is the chat watching them. I was the narrative designer and tools programmer. Released on itch.io.

Status. The chat system, the viewership meter, the dare system, the phone signal model and the node editor all of this is authored in are shipped, and the Zone A version of this dare pattern is wired and playtested in the released build. This particular quest is designed, not built. It was cut for time and uses only nodes that already run. The hide volume and the entity AI are a teammate's code; the quest, the chat, the dialogue and the tool are mine.

1. QUEST AT A GLANCE

Quest ID	dead_air
Zone	C, the 1950s bunker under the theatre
Type	Story quest, required to progress
Objective	Restore power to the bunker, then survive what the noise wakes up
The choice	How long the player is willing to stay hidden
Reward	Power, signal, and the biggest viewership spike in the zone
Cost	The entity learns the bunker is occupied, and the generator does not survive the night
Play time	6 to 8 minutes

2. THE DESIGN PROBLEM

The game runs on one meter. Viewership is the player's health bar, so at zero the stream ends and the run is over. Viewership is also what makes the monster dangerous, since the entity's awareness scales with the size of the audience. A big crowd keeps the player alive and it gets the player found.

Zone C is where I wanted that tension to become an active decision. The zone opens by taking the chat away. Climbing down the trapdoor drops the phone to no signal, so the chat panel goes to static, the viewer count greys out and no messages render. The audience is still there and still draining. The player just cannot hear them.

So the player arrives wanting the signal and the audience back. The quest gives them both, immediately and generously, then makes the player pay for it in a way they choose themselves.

The hiding spot has to work. Horror games train players that hiding is the correct answer: get in the locker, wait, the monster leaves. Here the hiding spot is genuinely safe from the monster and lethal anyway, because nobody watches a stream of someone crouching in the dark. The longer the player stays hidden the safer they get from the entity and the closer they get to losing the audience. The quest resolves when the player decides they are more frightened of an empty chat than of the thing in the corridor.

3. BEAT STRUCTURE

Beats are written in a fixed shape so they stay implementable: **beat, trigger, chat, dare, outcome**. This is the format the rest of the team builds from.

Phase 1: Restore the power

C.3.a: The generator room

- **Trigger:** enter_generator_room (proximity volume)
- **Chat:** suppressed at dispatch. Signal is None.
- **Read:** the room is rusted through except the generator, which is not. Someone kept this thing serviced. Mira Talon's log is pinned to the housing at eye height, and she is one of the staff whose notes the player has been reading since the ladder.
- **Outcome:** the objective is legible without a marker. It is the only working machine in a dead room and the player wants the signal back.

C.3.b: Getting it running

- **Trigger:** interact_generator
- **Outcome:** signal jumps to **High**, static clears, the viewer count colours back in and spikes higher than anywhere else in the zone. The chat returns all at once and it is thrilled.
- **Pacing:** the only unambiguously good moment in Zone C. Should last about four seconds.

C.3.c: What the noise woke

- **Trigger:** event_generator_on, then a short lead in to entity_going_generator

- **Chat:** the payoff pool is still running when the first noise lands, so celebration and dread overlap for a beat.

Phase 2: Dead air

C.3.d: The chase

- **Trigger:** entity_going_generator starts the entity_chase flood
- **Chat:** one line every 0.2 to 0.6 seconds, all telling the player to hide. The audience is, for the only time in the game, unanimous and correct.

C.3.e: The turn

- **Trigger:** player_hidden
- **Chat:** the flood stops, two lines of relief, then nothing.
- **Underneath:** the boredom counter starts. Every few seconds the chat gets less patient and the drain gets steeper. The entity is calming down at the same time, because its awareness is tied to the viewer count and the viewer count is falling.
- **What the player learns:** waiting works on the monster and it is killing the stream.

C.3.f: The dare

- **Trigger:** issued from inside hiding at boredom stage two
- **Dare:** peek_from_hiding. Banner: SHOW US
- **Asks:** break cover long enough to put the entity on camera. Gives the player a way to buy viewers without committing to leaving, so the decision has a middle setting.

C.3.g: Coming out

- **Trigger:** player_unhidden
- The payoff scales with the risk. With the entity still close the audience jumps hard. If the player waited until the corridor was clear the chat is small, flat and faintly disappointed, and the viewer count never recovers. The game pays for danger and it does not hide that it does.

C.3.h: Resolution

- The entity destroys the generator on its way out. Signal drops to **Low** rather than back to None. The player ends up permanently worse off than before turning it on, and better off than in the dark. That is the trade they made.

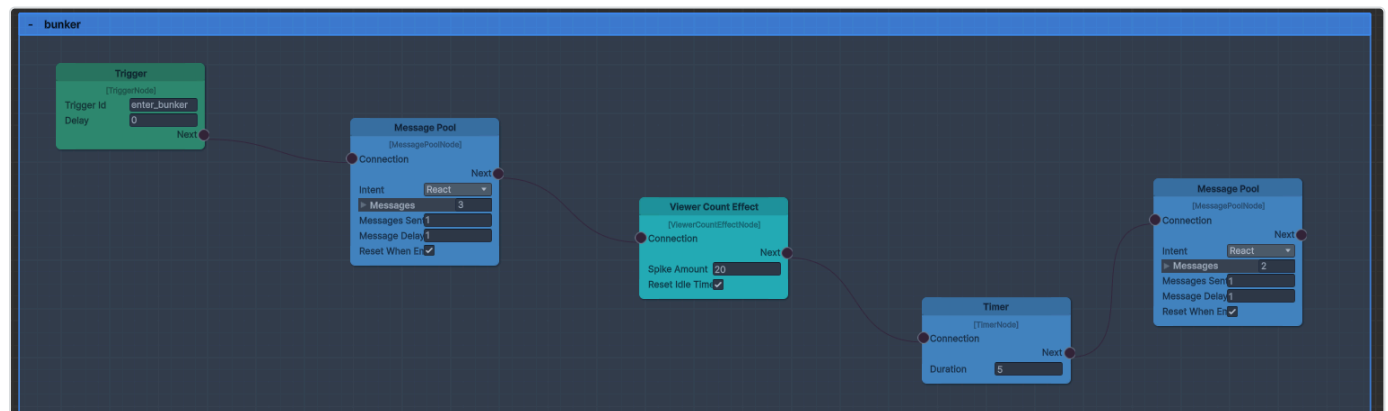
Failure states

- **Physical:** caught in the open. Downed, stream cuts, run ends.
- **Social:** viewership reaches zero while hidden. The stream ends with the player perfectly safe, crouched in a cupboard, alone. This is the failure that matters. The safe option was losing the whole time.

4. DIALOGUE TREE

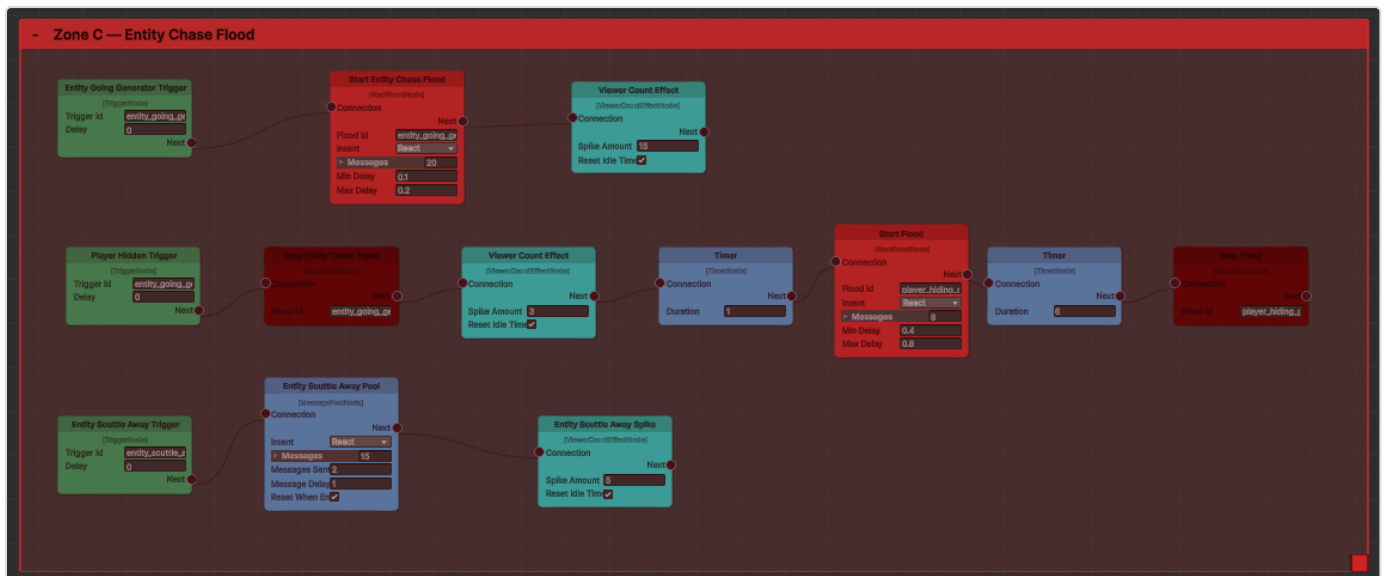
All of this is authored in ChatGraph, the node based dialogue and quest editor I built for the game. The graphs below are screenshots of the shipped project, not diagrams drawn for this document.

Phase 1 shape, from the shipped bunker entry



Shipped. A trigger fires a chat pool, the pool raises the viewer count, a timer holds, and a second pool comes back in a changed tone. Dead Air's Phase 1 is this chain with interact_generator as the trigger, a larger spike on the signal returning, and the second pool carrying unease instead of arrival.

Phase 2, the Zone C chase and hide graph



Shipped. Three chains in one group. The entity approach starts a 20 message flood at 0.1 to 0.2 second spacing, which is the panic. The player hiding stops that flood, pays out a small spike, waits a beat, then opens a slower 8 message flood at 0.4 to 0.8 while the player is in cover. The entity leaving runs its own pool and spike. Dead Air keeps this skeleton and adds what section 3 describes.

What Dead Air adds on top

The chains above run today. The boredom staging and the dare are the new work, written against node types that already exist in the tool.

```
CounterNode(boredom_stage) <- ticks while player_hidden holds
| 1 → MessagePoolNode ["restless"] + ViewerCountEffectNode(-5)
| 2 → MessagePoolNode ["prodding"] + ViewerCountEffectNode(-10)
|   └─ DareNode(peek_from_hiding)
|       └─ timelimit 20, banner "SHOW US"
|           └─ isStoryChallenge false, retryUntilSuccess false
|               └─ onComplete → ViewerCountEffectNode(+30) → MessagePool ["thrill"]
|                   └─ onFail → ViewerCountEffectNode(-10) → MessagePool ["coward"]
| 3 → MessagePoolNode ["leaving"] + ViewerCountEffectNode(-20)
| 4+ → StartFloodNode(exodus) + ViewerCountEffectNode(-30)

TriggerNode(player_unhidden)
└─ StopFloodNode(exodus)
    └─ FlagConditionNode(flag_entity_nearby)
        └─ true → ViewerCountEffectNode(+25) → MessagePoolNode ["they saw it"]
            └─ false → ViewerCountEffectNode(+5) → MessagePoolNode ["flat", 2 msgs]
```

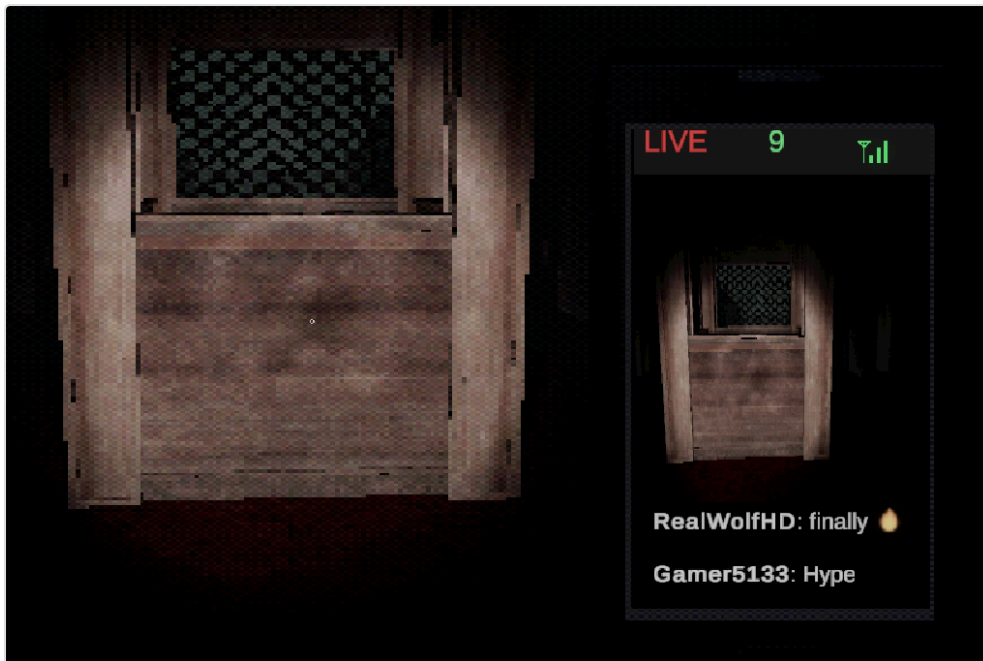
The chase flood hangs off the entity, not the generator. Wired to `interact_generator` the panic would start the instant the player pulled the lever, and they would learn that the lever makes the noise. Hanging it off the entity trigger lets the AI own the lead in, so there are a few seconds where the signal is back and everything is fine, and then the gap closes. The shipped graph is already built this way.

Boredom is a counter rather than a timer. A flat drain is just a number going down. Staging it means the chat's tone escalates in steps the player can hear, so they feel how much patience is left without reading the meter. It also gives the dare a natural spot at stage two, when the audience is annoyed but has not started leaving.

5. DIALOGUE

Two constraints govern every line.

The audience can only perceive what the camera and microphone pick up. They cannot smell the room, do not know what is behind the player and have no access to anything off screen. Chat reacts to what it can see, a hesitation, an unopened door, visible decay, or what it can hear on the mic. That matters more here than anywhere else in the game, because for most of Phase 2 the camera is pointed at the inside of a cupboard. The audience genuinely cannot see the thing hunting the player, so their boredom is sincere.



Shipped build. The phone is the stream: live indicator, viewer count, signal bars and the last chat lines, all on screen at once. The audience only ever sees what the camera on that phone is pointed at, which is the constraint every line in this section is written against. The count keeps moving whether anything is happening or not.

Chat is in Phase 2 of a three phase voice guide I wrote: ordinary, unsettling, wrong. Phase 2 chat has started knowing things slightly before it should, just enough to be off.

Payoff, on the generator starting

LETS GOOO {emoji:positive}[2-4]
LIGHTS
signal's back, we can hear you
okay that was worth it

Unease, 2.5 seconds later

whats that
mute a sec
no keep it on

Chase flood (the pool already in the shipped graph)

oh no HIDE HIDE NOW RUN NO DONT RUN HIDE its coming IT'S COMING something is moving holy shit behind you get out get out go
go go @{} HIDE BRO i can hear it dont look at it why did you start it oh no no no holy {emoji:fearful}[2-3]

Relief, on hiding

okay okay good
dont move

Boredom 1, restless

is it gone
how long are we doing this
i cant see anything

Boredom 2, prodding, and the dare

bro
we came for a stream not a cupboard
just look. one look
@{} SHOW US

Boredom 3, leaving

this is dead air
nothing is happening
mods can someone clip the good part, im out
she was right lol. shouldve left it off

That last line is the Phase 2 tell. Nobody in chat read Mira's note. The player read it alone, with no signal, eleven minutes earlier.

Exodus flood

im out same gn stream's over unfollowing wake me when he moves {emoji:negative}[2-3]

Coming out with the entity still close

WAIT
THERE
ITS RIGHT THERE
clip that clip that clip that

Coming out after the corridor is clear

oh. its gone
cool

Two lines, one of them "cool". The flatness is the punishment for playing it too safe.

Aftermath

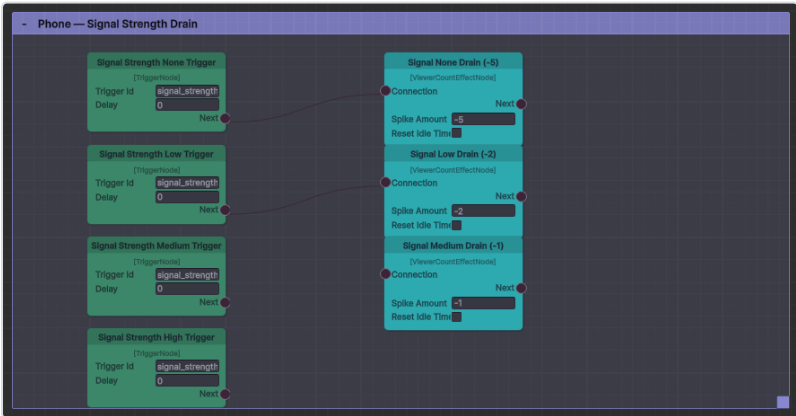
that was your fault btw

6. STATES, FLAGS AND TRIGGERS

Flag	Set by	Read by
flag_generator_on	SetFlagNode on generator interact	entity activation, signal state, Zone C→D gate
flag_entity_nearby	entity AI proximity check	the player_unhidden branch
dare_active_peek_from_hiding	DareNode on arm	dare banner UI
dare_success_peek_from_hiding	camera-on-entity check	DareNode completion
boredom_stage	CounterNode while hidden	drain amount and chat tone

Trigger	Source	Status
enter_generator_room	ProximityChatTrigger	designed
interact_generator	Interactable	designed
entity_going_generator	entity AI on chase start	graph wired, awaiting AI hook
player_hidden	hide volume	graph wired, awaiting hide system
player_unhidden	hide volume, on exit	designed

Signal states this quest moves through: None on zone entry, High on success, Low after the entity destroys the generator. That model is shipped: SignalManager, PhoneSignalUI, a hand written static shader, and the drain ladder below.



Shipped. One trigger per signal level. No signal costs 5 viewers a tick, low costs 2, medium costs 1, and full signal has no drain wired to it at all. Signal is not set dressing on the phone, it is a running cost, which is what makes restoring the generator worth walking into the dark for.

7. TUNING

The quest lives or dies on one relationship: how fast the boredom drain runs against how long the entity takes to lose interest.

- Drain too slow and hiding is correct, so the quest becomes a waiting room.
- Drain too fast and the player is shoved out of cover before understanding why, which reads as the game being unfair rather than the audience being fickle.

Starting numbers are the ones in the tree: roughly 40 to 50 viewers lost over about 30 seconds of hiding, against an entity that takes 20 to 25 seconds to wander off. That puts the safe exit slightly after the drain starts to hurt, so the correct play is to come out a little early and take a real risk.

I would not defend those numbers before watching six people play through. The shape I would defend: the safe option has to be losing, and the player has to work that out themselves.

8. IMPLEMENTATION

Authored in **ChatGraph**, the node based dialogue and quest editor I built in Unity on top of the open source xNode. Graphs and nodes are ScriptableObject assets, so a quest is a file a writer can open, rewire and save without touching code or waiting on a programmer. Fifteen authorable node types exist. This quest uses TriggerNode, MessagePoolNode, CooldownNode, TimerNode, CounterNode, DareNode, FlagConditionNode, SetFlagNode, ViewerCountEffectNode, StartFloodNode and StopFloodNode.

In the scene, the generator is an Interactable carrying two data driven actions: one sets the success flag, one is a SetSignalStrengthAction moving the phone to High. Both drop into any interactable from the inspector, so putting this quest in the level is content work rather than a code change.

The hide volume and the entity AI are my teammate's systems. This quest consumes them through two triggers and one flag, on purpose, so the narrative layer depends on gameplay systems through a thin named interface and either side can be rebuilt without renegotiating the other.

9. PROCESS

Started from a hole in the level. The bunker existed before this quest did, and playtesting exposed the problem: taking the chat away also removed the only thing pushing the player forward. The zone went quiet and needed a reason to make noise again.

Used a thing the player already wanted. The signal was an existing system, visible and already irritating, which made it the natural currency. A reward carries more weight when the game has spent ten minutes making the player feel its absence, and setting that up cost nothing to build.

Reward and threat had to be the same event. Turning the generator on gives the player their audience back and tells the entity where they are. Collapsing those into one action turns a task into a decision, so the player is paying for exactly what they asked for.

Inverted the safe answer, which was the design work that mattered. The first version ended with the player hiding until the monster left. It played fine and was forgettable, because every horror game has already taught players to do that. The fix was to make the hiding place work perfectly and cost the player the thing they were protecting. Once the audience drains while the player is safe, every second in the cupboard is a decision.

The warning has a name on it. The note on the generator is signed by Mira, whose logs the player has been reading for ten minutes. An anonymous DANGER sign is set dressing. A specific frightened person telling the player not to costs one line of text and buys a character.

The perception rule found the shape. An early draft had chat reacting to the entity's movements while the player was hidden. I cut those lines because the camera was pointed at a cupboard door and the audience could not possibly see the entity. Losing them turned the hiding section from a horror scene into a boredom scene, which was what made the quest work.

Wrote down the number I am unsure about. The drain rate sits in my open questions with both arguments and a starting value, rather than getting quietly settled by whatever I first typed into an inspector. On a team, the things a designer has not resolved are the most useful things to write down.